

本节例题列表

本节讲解的例题及其囊括的知识点，如表4-7所示。

表4-7 动态树与LCT例题归纳

编 号	题 号	标 题	知 识 点	代 码 作 者
例 4-33	NC 20311	Cave	LCT; 维护连通性	陈锋
例 4-34	UVa 11994	Happy Painting	LCT; 维护路径信息	魏子豪、陈锋
例 4-35	CodeChef-GERALD 07	Chef and Graph Queries	LCT; 维护 MST	陈锋

4.6 离线算法

离线算法使用得当，可以大幅度简化一些本来需要复杂数据结构来解决的问题，本节提供了基于时间分治、整体二分、莫队的经典题目的代码实现。

例4-36 【基于时间分治；三维偏序】动态逆序对（CQOI 2011，牛客NC 19919）

对于序列 A ，它的逆序对数定义为满足 $i < j$ ，且 $A_i > A_j$ 的数对 (i, j) 的个数。给出 $1 \sim N$ ($N \leq 10^5$) 的一个排列，按照某种顺序依次删除其中的 M 个元素，在每次删除一个元素之前统计整个序列的逆序对数。

【代码实现】

```

// 陈锋
#define _for(i, a, b) for (int i = (a); i < (int)(b); ++i)
#include <bits/stdc++.h>
using namespace std;
typedef long long LL;
template <int SZ>
struct BIT {
    int C[SZ], N;
    inline int lowbit(int x) { return x & -x; }
    void add(int x, int v) {
        while (x <= N) C[x] += v, x += lowbit(x);
    }
    int sum(int x) {
        int r = 0;
        while (x) r += C[x], x -= lowbit(x);
        return r;
    }
};
const int NN = 1e5 + 8;
BIT<NN> S;
struct OP {
    int id, p, v; // 第id个命令, 在位置p插入v
    bool operator<(const OP &b) { return p > b.p; }
};

```

```

} O[NN], T[NN];
int N, M, A[NN], Pos[NN], Vis[NN];
LL Ans[NN]; // 第id个命令插入x之后增加多少个逆序对
void solve(int l, int r) { // 按照插入位置从大到小排序
    if (l == r) return;
    int m = (l + r) / 2, l1 = l, l2 = m + 1;
    for (int i = l; i <= r; i++) { // 情况1
        const OP &o = O[i];
        if (o.id <= m) S.add(o.v, 1); // id ∈ [l,m]
        else Ans[o.id] += S.sum(o.v); // id ∈ [m + 1,r]
    }
    for (int i = l; i <= r; i++) if (O[i].id <= m) S.add(O[i].v, -1); // 还原BIT
    for (int i = r; i >= l; --i) { // 情况2
        const OP &o = O[i];
        if (o.id <= m) S.add(N - o.v + 1, 1); // id ∈ [l,m], 记录插入的  $N - v + 1 \geq v$  的元素个数
        else Ans[o.id] += S.sum(N - o.v + 1);
        // id ∈ [m + 1,r], v映射到  $N - v + 1$ , 如  $N \rightarrow 1, N - 1 \rightarrow 2$ 
    }

    for (int i = l; i <= r; i++) { // 分治: 把 id ∈ [l,m], [m+1,r] 的操作分别放两边
        const OP &o = O[i];
        if (o.id <= m) T[l1++] = o, S.add(N - o.v + 1, -1); // 还原BIT
        else T[l2++] = o;
    }
    copy(T + l, T + r + 1, O + l);
    solve(l, m), solve(m + 1, r);
}

int main() {
    cin >> N >> M;
    int id = N, qc = M;
    S.N = N;
    for (int i = 1; i <= N; ++i) cin >> A[i], Pos[A[i]] = i;
    for (int i = 1; i <= M; ++i) {
        OP &q = O[i];
        cin >> q.v, Vis[q.p = Pos[q.v]] = true, q.id = id--;
    }
    for (int i = 1; i <= N; ++i) {
        if (Vis[i]) continue;
        O[++qc] = {id--, i, A[i]};
    }
    sort(O + 1, O + 1 + N); // 根据插入位置递减排序
    solve(1, N);
    for (int i = 1; i <= N; ++i) Ans[i] += Ans[i - 1];
    _for(i, 0, M) cout << Ans[N - i] << endl;
    return 0;
}
/*

```

算法分析请参考：《算法竞赛入门经典——训练指南》升级版 3.9 节例题 45

同类题目：Sherlock and Inversions, CodeChef IITI 15

Mokia, BOI 2007, 牛客 NC 51145

*/

例4-37 【基于时间分治优化DP; NTT】公交路线 (Bus Routes, ACM/ICPC合肥2015,HDU 5552)

给出 N ($1 \leq N \leq 10\,000$) 个点以及 M ($0 < M < 2^{31}$) 种颜色。要构造一个无向连通图，每条边必须选择一种颜色，并且这个连通图至少包含一个环。计算共有多少种这样的构图方案，输出方案数模152 076 289的值。

【代码实现】

```

// 陈锋
#include <bits/stdc++.h>
#define _for(i, a, b) for (int i = (a); i < (int)(b); ++i)
#define _rep(i, a, b) for (int i = (a); i <= (int)(b); ++i)
using namespace std;
typedef long long LL;
const int MOD = 152076289, NN = 10000 + 8;
LL gcd(LL a, LL b) { return b ? gcd(b, a % b) : a; }
void exgcd(LL a, LL b, LL &x, LL &y) {
    if (!b) {
        x = 1, y = 0;
        return;
    }
    exgcd(b, a % b, y, x), y -= a / b * x;
}
inline LL mul_mod(LL a, LL b) { return a * b % MOD; }
inline LL pow_mod(LL a, LL p) {
    LL res = 1;
    for (; p > 0; p >>= 1, (a *= a) %= MOD)
        if (p & 1) (res *= a) %= MOD;
    return res;
}
inline LL inv(LL a) {
    LL x, y;
    exgcd(a, MOD, x, y);
    return (x % MOD + MOD) % MOD;
}

namespace _Polynomial {
    const int g = 106; // 原根
    int A[NN << 1], B[NN << 1];
    int w[NN << 1], r[NN << 1];
    void DFT(int *a, int op, int n) {
        _for(i, 0, n) if (i < r[i]) swap(a[i], a[r[i]]);
        for (int i = 2; i <= n; i <<= 1)
            for (int j = 0; j < n; j += i)

```

```

    for (int k = 0; k < i / 2; k++) {
        int u = a[j + k],
            t = (LL)w[op == 1 ? n / i * k : (n - n / i * k) & (n - 1)] *
                a[j + k + i / 2] % MOD;
        a[j + k] = (u + t) % MOD, a[j + k + i / 2] = (u - t) % MOD;
    }
    if (op == -1) {
        int I = inv(n);
        _for(i, 0, n) a[i] = (LL)a[i] * I % MOD;
    }
}

void multiply(const int *a, const int *b, int *c, int n1, int n2) {
    int n = 1;
    while (n < n1 + n2 - 1) n <<= 1;
    copy_n(a, n1, A), copy_n(b, n2, B);
    fill(A + n1, A + n, 0), fill(B + n2, B + n, 0);

    _for(i, 0, n) r[i] = (r[i] >> 1 >> 1) | ((i & 1) * (n >> 1));
    w[0] = 1, w[1] = pow_mod(g, (MOD - 1) / n);
    _for(i, 2, n) w[i] = mul_mod(w[i - 1], w[1]);

    DFT(A, 1, n), DFT(B, 1, n);
    _for(i, 0, n) A[i] = mul_mod(A[i], B[i]);
    DFT(A, -1, n);
    _for(i, 0, n1 + n2 - 1) c[i] = (A[i] + MOD) % MOD;
}

}; // namespace _Polynomial

int A[NN], B[NN], C[NN * 2];
LL Fact[NN], FactInv[NN], F[NN], G[NN];
void solve(int l, int r) {
    if (l == r) {
        F[l] = (G[l] - mul_mod(Fact[l - 1], F[l])) % MOD;
        return;
    }
    int m = (l + r) / 2;
    solve(l, m); // F[l~m] -> F(m, r)
    _rep(i, 1, m) A[i - 1] =
        mul_mod(F[i], FactInv[i - 1]); //  $\sum F(i)/(i-1)!, i = 1 \sim m$ 

    for (int i = r - 1, j = 0; i >= 1; --i, ++j)
        B[j] = mul_mod(G[r - i], FactInv[r - i]);
    _Polynomial::multiply(A, B, C, m - 1 + 1, r - 1);
    _rep(i, m + 1, r) (F[i] += C[i - 1 - 1]) %= MOD;
    solve(m + 1, r);
}

int main() {
    Fact[0] = Fact[1] = 1, FactInv[0] = FactInv[1] = 1; // i!, (i!)^{-1} % MOD

```

```

_for(i, 2, NN) Fact[i] = mul_mod(Fact[i - 1], i), FactInv[i] = inv(Fact[i]);
LL m;
int T;
scanf("%d", &T);
for (int t = 1, n; t <= T; t++) {
    scanf("%d%lld", &n, &m);
    fill_n(F, n + 1, 0);
    _rep(i, 1, n) G[i] = pow_mod(m + 1, (LL)i * (i - 1) / 2);
    solve(1, n);
    printf("Case #%d: %lld\n", t,
        ((F[n] - pow_mod(n, n - 2) * pow_mod(m, n - 1)) % MOD + MOD) % MOD);
}
return 0;
}
/*
算法分析请参考:《算法竞赛入门经典——训练指南》升级版 3.9 节例题 46
同类题目: Cash, NOI 2007, 牛客 NC 17519
*/

```

例 4-38 【整体二分】流星 (Meteors, POI 2011, SPOJ METEORS)

有 n 个国家和 m 个空间站 ($1 \leq n, m \leq 3 \times 10^5$)，每个空间站都属于一个国家，一个国家可以有多个空间站。所有空间站按照顺序形成一个环，也就是说， i 和 $i+1$ 号空间站相邻，而 m 号空间站和1号空间站相邻。现在，将会有 k ($1 \leq k \leq 3 \times 10^5$) 场流星雨降临，第 i 场流星雨会给区间 $[l_i, r_i]$ 内的每个空间站带来 a_i ($1 \leq a_i \leq 10^9$) 单位的陨石。如果 $l_i > r_i$ ，则区间 $[l_i, r_i]$ 指的是 $l_i, l_i+1, \dots, m, 1, 2, \dots, r_i-1, r_i$ 。每个国家都有一个收集陨石的目标 p_i ($1 \leq p_i \leq 10^9$)，即第 i 个国家需要收集 p_i 单位的陨石。计算每个国家最早完成陨石收集目标是在第几场流星雨过后。

【代码实现】

```

// 陈锋
#include <bits/stdc++.h>
#define _for(i,a,b) for(int i=(a); i<(int)(b); ++i)
#define _rep(i,a,b) for(int i=(a);i<=(b);++i)
using namespace std;
typedef long long LL;
template <int SZ>
struct BIT {
    LL C[SZ];
    int N;
    void init(int _n) { N = _n; }
    inline int lowbit(int x) { return x & -x; }
    inline void add(int x, int d) { while (x <= N) C[x] += d, x += lowbit(x); }
    inline LL sum(int x) {
        LL ret = 0;
        while (x) ret += C[x], x -= lowbit(x);
        return ret;
    }
}

```



```

};
struct Rain { int l, r, a; };
const int NN = 3e5 + 8;
Rain Rs[NN];
vector<int> St[NN]; // 每个国家的空间站
int N, M, Ans[NN], P[NN];
BIT<NN> S;
inline void apply(const Rain& q, bool revert = false) {
    int x = q.a, l = q.l, r = q.r;
    if (revert) x = -x;
    if (l <= r) S.add(l, x), S.add(r + 1, -x); // 区间加单点查询用差分实现
    else S.add(l, x), S.add(M + 1, -x), S.add(1, x), S.add(r + 1, -x); // 拆成两个区间
}
// C 中的每个国家的查询结果进行二分, 目标答案区间是[al, ar]
void solve(const vector<int>& C, int l, int r) {
    if (C.empty()) return;
    if (l == r) { // 答案的目标区间确定
        for (int c : C) Ans[c] = l;
        return;
    }
    int m = (l + r) / 2;
    _rep(ai, l, m) apply(Rs[ai]); // 看看[l,m]中下的流星雨够不够
    vector<int> LC, RC;
    for (int c : C) { // 每个国家都看看
        int &p = P[c];
        LL x = 0;
        for (int s : St[c]) if ((x += S.sum(s)) >= p) break; // 收集够了
        if (p <= x) LC.push_back(c); // 答案在[l,m]中, 国家分到左边
        else p -= x, RC.push_back(c); // 答案在[m+1,r]中, 国家分到右边
    }
    _rep(ai, l, m) apply(Rs[ai], true); // 看看[l,m]中下的流星雨够不够-还原
    solve(LC, l, m), solve(RC, m + 1, r); // 更改顺序, 整体二分
}

int main() {
    ios::sync_with_stdio(false), cin.tie(0);
    cin >> N >> M, S.init(M + 2);
    int qc, x;
    vector<int> C;
    _rep(i, 1, M) cin >> x, St[x].push_back(i);
    _rep(i, 1, N) cin >> P[i], C.push_back(i);
    cin >> qc;
    _rep(i, 1, qc) cin >> Rs[i].l >> Rs[i].r >> Rs[i].a; // 流星雨下到[l,r], 雨量为 a
    solve(C, 1, qc + 1);
    _rep(i, 1, N) {
        if (Ans[i] <= qc) cout << Ans[i] << endl;
        else cout << "NIE" << endl;
    }
    return 0;
}

```

```
}
/*
算法分析请参考：《算法竞赛入门经典——训练指南》升级版 3.9 节例题 47
同类题目：Arnook Defensive Line, SPOJ KL11B
*/
```

例4-39 【整体二分；树上差分】金币 (Coins, ACM/ICPC Asia - Amritapuri 2015, Codechef AMCOINS)

给出一棵包含 N ($1 \leq N \leq 5 \times 10^5$) 个结点的树，每个结点可以收集金币。然后给出 Q ($1 \leq Q \leq 10^5$) 个操作，操作分为以下两种类型。

- Give(X, Y, W): 给结点 X 到 Y 路径上的所有结点 (包含 X 和 Y) 一个面值为 W ($1 \leq W \leq 10^5$) 的金币。
- Find(Z, I, J, K): 找到结点 Z 所有在第 I 次和第 J 次操作 (包含 I 和 J) 之间获得的金币中第 K 小的金币面值。如果 Z 只有不到 K 个金币，则输出-1。注意，一个结点可能收到多个同样面值的金币。如果 Z 收到的金币是 $\{1, 2, 3, 3, 4\}$ ，则第2小的金币面值是2，第3小的金币面值是3，第4小的金币面值是3。

【代码实现】

```

// 陈锋
#include <bits/stdc++.h>
using namespace std;
template <int SZ>
struct BIT {
    int C[SZ], sz;
    void init(int _sz) {
        sz = _sz;
        assert(sz + 1 < SZ);
    }
    inline int lowbit(int x) { return x & -x; }
    void add(int x, int y) {
        while (x < SZ) C[x] += y, x += lowbit(x);
    }
    int sum(int x) {
        int s = 0;
        while (x > 0) s += C[x], x -= lowbit(x);
        return s;
    }
};

const int NN = 5e5 + 8, QQ = 1e5 + 8, HH = 20;
vector<int> G[NN];
struct Cmd {
    int op, x, y, w, z, k, id, time;
    friend bool operator<(const Cmd& a, const Cmd& b) {
        if (a.time != b.time) return a.time < b.time;
        return a.op < b.op;
    }
};

```

```

int Tin[NN], Tout[NN], Dfn, Fa[NN][HH + 1], Dep[NN]; // DFS, LCA
int lca(int u, int v) {
    if (Dep[u] < Dep[v]) swap(u, v);
    int d = Dep[u] - Dep[v];
    for (int h = 0; h <= HH; h++)
        if (d & (1 << h)) u = Fa[u][h];
    if (u == v) return u;
    for (int h = HH; h >= 0; h--)
        if (Fa[u][h] != Fa[v][h]) u = Fa[u][h], v = Fa[v][h];
    return Fa[u][0];
}
void dfs(int u, int fa) { // Tin[u]: 先序遍历序列中的编号
    Tin[u] = ++Dfn, Fa[u][0] = fa, Dep[u] = Dep[fa] + 1;
    for (int h = 1; h <= HH; h++) Fa[u][h] = Fa[Fa[u][h - 1]][h - 1];
    for (auto v : G[u])
        if (fa != v) dfs(v, u);
    Tout[u] = Dfn; // Tin[u]-Tout[u]: u 子树先序遍历序列中的区间
}
BIT<NN> S;
int Cnt[QQ], Ans[QQ];
void apply(const Cmd& q, bool rev = false) {
    int d = lca(q.x, q.y), c = rev ? -1 : 1; // x-y 路径上全部增加一个计数
    S.add(Tin[q.x], c), S.add(Tin[q.y], c), S.add(Tin[d], -c);
    // +(x-root), +(y-root), -(d-root)
    if (d != 1) S.add(Tin[Fa[d][0]], -c); // d != root, -(fa(d)-root)
}
void solve(int al, int ar, const vector<Cmd>& qs) { // Qs[ql,qr] 答案在 [al,ar] 中
    if (qs.empty()) return;
    int am = (al + ar) / 2;
    vector<Cmd> B;
    for (const auto& q : qs) {
        if (q.op == 1) { // 修改操作
            if (q.w <= am) B.push_back(q); // 增加一个 [al, am] 中的金币
        } else { // query[], 拆成对两个时间段的查询: [1, i-1], [1, j], 结果考虑正负
            B.push_back(q), B.back().time = q.x - 1, B.back().w = -1;
            B.push_back(q), B.back().time = q.y, B.back().w = 1;
            Cnt[q.id] = 0; // [al, am] 中的操作在 q.z 结点增加了几个金币
        }
    }
    sort(begin(B), end(B)); // 时间排序, 相同时间, 写在读前
    for (const Cmd& q : B) {
        if (q.op == 1)
            apply(q); // 修改操作, 树上差分
        else
            // 版本 [1, j] 中的增加的 [al, am] 中的金币数量减去 [1, i-1] 中增加的 [al, am] 中的金币数量
            Cnt[q.id] += q.w * (S.sum(Tout[q.z]) - S.sum(Tin[q.z] - 1));
    }
    for (const Cmd& q : B)
        if (q.op == 1) apply(q, true); // 还原所有修改操作
}

```

```

if (al == ar) { // 答案已经锁定
    for (auto& q : qs)
        if (q.op == 2 && Cnt[q.id] >= q.k) Ans[q.id] = al;
    return;
}
vector<Cmd> lqs, rqs;
for (auto& q : qs) {
    if (q.op == 1) {
        if (q.w <= am) lqs.push_back(q); // 插入一个[al,am]中的金币
        else rqs.push_back(q); // 插入一个[am+1,ar]中的金币
    } else {
        if (Cnt[q.id] >= q.k) lqs.push_back(q); // 答案在[al,am]中的查询
        else rqs.push_back(q), rqs.back().k -= Cnt[q.id]; // 答案在[am+1,ar]中的查询
    }
}
solve(al, am, lqs), solve(am + 1, ar, rqs);
}
int main() {
    ios::sync_with_stdio(false), cin.tie(0);
    int n, m, qc = 0;
    cin >> n;
    for (int i = 1, x, y; i < n; i++)
        cin >> x >> y, G[x].push_back(y), G[y].push_back(x);
    Dfn = 0, dfs(1, 0);
    cin >> m;
    vector<Cmd> Qs(m);
    for (int i = 1; i <= m; i++) {
        Cmd& q = Qs[i - 1];
        cin >> q.op;
        if (q.op == 1)
            cin >> q.x >> q.y >> q.w, q.time = i;
        else
            cin >> q.z >> q.x >> q.y >> q.k, q.id = ++qc;
    }
    solve(1, 100000, Qs);
    for (int i = 1; i <= qc; i++) printf("%d\n", Ans[i] ? Ans[i] : -1);
    return 0;
}
/*
算法分析请参考:《算法竞赛入门经典——训练指南》升级版 3.9 节例题 48
同类题目: 混合果汁, CTSC 2018 Day2, 牛客 NC 200498
*/

```

例4-40 【莫队】D-查询 (D-query, SPOJ DQUERY)

给出一个数组 $a_1, a_2, \dots, a_n$ 以及 M 个 d -查询 (i, j) ($1 \leq i \leq j \leq n$)。对于每个 d -查询 (i, j) ，返回数组子序列 $a_i, a_{i+1}, \dots, a_j$ 中的不同元素个数。

【代码实现】

```

// 陈锋
#include <bits/stdc++.h>

using namespace std;
#define _for(i, a, b) for (int i = (a); i < (int)(b); ++i)
#define _rep(i, a, b) for (int i = (a); i <= (int)(b); ++i)

const int NN = 300000 + 4, MM = 200000 + 4, AA = 1000000 + 4;
int A[NN], ANS[MM], N, M, BLOCK;
struct query {
    int L, R, id;
    bool operator<(const query& q) const {
        int lb = L / BLOCK;
        if (lb != q.L / BLOCK) return lb < q.L / BLOCK;
        if (lb % 2) return R < q.R;
        return R > q.R;
    }
};

query Q[MM];
int ans, curL, curR, CNT[AA];
void add(int pos) { if (++CNT[A[pos]] == 1) ++ans; }
void remove(int pos) { if (--CNT[A[pos]] == 0) --ans; }
typedef long long LL;
int main() {
    scanf("%d", &N);
    _rep(i, 1, N) scanf("%d", &A[i]), CNT[A[i]] = 0;
    scanf("%d", &M);
    BLOCK = max((int)ceil((double)N / sqrt(M)), 16);
    _for(i, 0, M) scanf("%d%d", &Q[i].L, &Q[i].R), Q[i].id = i;
    sort(Q, Q + M);
    CNT[A[1]] = 1, ans = 1, curL = 1, curR = 1;
    _for(i, 0, M) { // 通过自增和自减操作符来简化代码
        while (curL < Q[i].L) remove(curL++);
        while (curL > Q[i].L) add(--curL);
        while (curR < Q[i].R) add(++curR);
        while (curR > Q[i].R) remove(curR--);
        ANS[Q[i].id] = ans;
    }
    _for(i, 0, M) printf("%d\n", ANS[i]);
    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典——训练指南》升级版 3.9 节例题 49
同类题目：Count on a tree II, SPOJ COT2
*/

```

例4-41 【带修改莫队】数颜色（牛客NC 202003）

墨墨购买了 N 支彩色画笔（其中有些颜色可能相同），摆成一行，你需要回答墨墨的提问。墨墨会向你发布如下指令。

□ Q $L R$: 询问从第 L 到第 R 支画笔中共有多少种不同颜色的画笔。

□ R $P Col$: 把第 P 支画笔替换为颜色 Col 。

为了满足墨墨的要求，你知道你需要干什么吗？

【代码实现】


```

// 陈锋
#include <bits/stdc++.h>

using namespace std;
#define _for(i, a, b) for (int i = (a); i < (int)(b); ++i)
#define _rep(i, a, b) for (int i = (a); i <= (int)(b); ++i)

const int SZ = 10005, MAXC = 1e6 + 4;
int BLOCK, Color[SZ], CurColor[SZ], CNT[MAXC], Ans[SZ];
struct Query {
    int l, r, id, c;
    bool operator<(const Query &rhs) const {
        if (l / BLOCK == rhs.l / BLOCK) {
            if (r / BLOCK == rhs.r / BLOCK) return id < rhs.id; // 时间维度优化
            return r < rhs.r;
        }
        return l < rhs.l;
    }
};
struct Change {
    int pos, old_color, color; // 位置, 旧颜色, 新颜色
    void apply();
    void revert();
};
Query Q[SZ];
Change Changes[SZ];
int curAns, curL, curR;
void add_pos(int a) {
    if (++CNT[a] == 1) curAns++;
}
void del_pos(int a) {
    if (--CNT[a] == 0) curAns--;
}
void Change::apply() { // 修改位置在当前区间内, 应用修改到结果中
    if (curL <= pos && pos <= curR) del_pos(old_color), add_pos(color);
    Color[pos] = color; // 应用修改
}
void Change::revert() { // 修改位置在当前区间内, 还原结果中的答案

```

```

    if (curL <= pos && pos <= curR) del_pos(color), add_pos(old_color);
    Color[pos] = old_color;          // 应用还原
}

int main() {
    int N, M, c1 = 0, c2 = 0;
    cin >> N >> M;
    BLOCK = pow(N, 2.0 / 3.0);
    _rep(i, 1, N) cin >> Color[i], CurColor[i] = Color[i];
    char opt[4];
    _rep(i, 1, M) {
        cin >> opt;
        if (opt[0] == 'Q') {
            Query &q = Q[c1];
            cin >> q.l >> q.r, q.id = c1++, q.c = c2;
        } else {
            Change &ch = Changes[c2++];
            cin >> ch.pos >> ch.color;
            ch.old_color = CurColor[ch.pos], CurColor[ch.pos] = ch.color;
        }
    }
    sort(Q, Q + c1);
    curL = 1, curR = 1, curAns = 0;
    int last_c = 0;                  // 第一条还未执行的修改命令编号
    add_pos(Color[1]);

    _for(i, 0, c1) {
        while (last_c < Q[i].c) Changes[last_c++].apply();
        // 应用在此查询时间之前的命令
        while (last_c > Q[i].c) Changes[--last_c].revert();
        // 回退在此查询时间之后的命令
        while (curR < Q[i].r) add_pos(Color[++curR]);
        while (curR > Q[i].r) del_pos(Color[curR--]);
        while (curL > Q[i].l) add_pos(Color[--curL]);
        while (curL < Q[i].l) del_pos(Color[curL++]);
        Ans[Q[i].id] = curAns;
    }
    _for(i, 0, c1) cout << Ans[i] << endl;
    return 0;
}
/*
算法分析请参考:《算法竞赛入门经典——训练指南》升级版 3.9 节例题 50
同类题目: Game, HDU 6610
*/

```

本节例题列表